第18章 动态规划DP

18.1 线性DP

18.1.1 线性DP的定义

线性DP是动态规划问题中的一类问题，指状态之间有线性关系的动态规划问题

18.1.2 线性DP的特点

- 状态描述的是一个线性的范围
- 转移比较简单，每个状态依赖的状态比较少，在线性结构上进行
-

通过一些例题来看看线性DP如何解题。

例题18-1：导弹拦截（zhima 781 NOIP 2010 普及组）

【题目描述】

有个特殊的导弹拦截系统有一个缺陷：虽然它的第一发炮弹能够到达任意的高度，但是以后每一发炮弹都不能高于前一发的高度。输入导弹依次飞来的高度，计算

这套系统最多能拦截多少导弹，导弹数量最多 1000 。

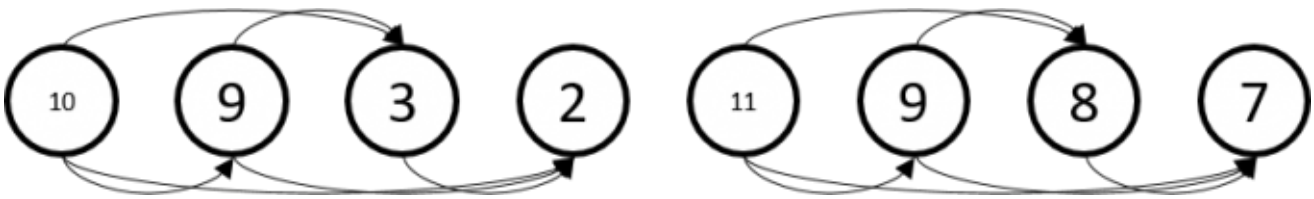
【样例输入】

```
389 207 155 300 299 170 158 65
```

【样例输出】

```
6
```

【题目分析】



弹道拦截的本质题意就是在一行整数中，找一个最长的不上升子序列，这类问题非常适合用动态规划（以下简称DP）来做，需要建立一个数组表示状态，状态这

一概念也是DP区别于搜索的其中一个要素，在线性DP问题中，状态往往是一个数组，它用来存放到目前为止的最优值，此题我们可以进行如下设计：

状态：$dp[i]$ 表示到第 i 发导弹为止，能够连成的最长数量，上图所表示的8个数中，最终解应该是 5 ，对应的序列就是 $10\ 9\ 9\ 8\ 7$ ，这个最优解的状态应该看到最

后一个 7 所在的位置 第 8 个导弹，用 $dp[8]$ 来表示，而 $dp[8]$ 的来源是 $dp[1]$ 、$dp[2]$ 、$dp[5]$ 、$dp[6]$ 、$dp[7]$ 中的最大值，对应上图带箭头的线。

状态转移：$dp[i] = max(dp[i], dp[j] + 1)(1 \le j \le i - 1)$

初值：$dp[i] = 1$，初值的设定非常重要，不然完美的状态及转移也将得不到正确的结果，这里为什么要设置初值 1 ？因为存在一个导弹，它的前面高度都比他

低，也就是说它没有成功地进行过状态转移过，这时候 $dp[i]$ 就应该是它本身长度的 1 。

终值：$max\{dp[i]\}(1 \le i \le n)$

时间复杂度：$O(n^2)$，线性DP的时间复杂度可以用状态数量 $\times$ 状态转移操作数，因此就是 $n \times n$。

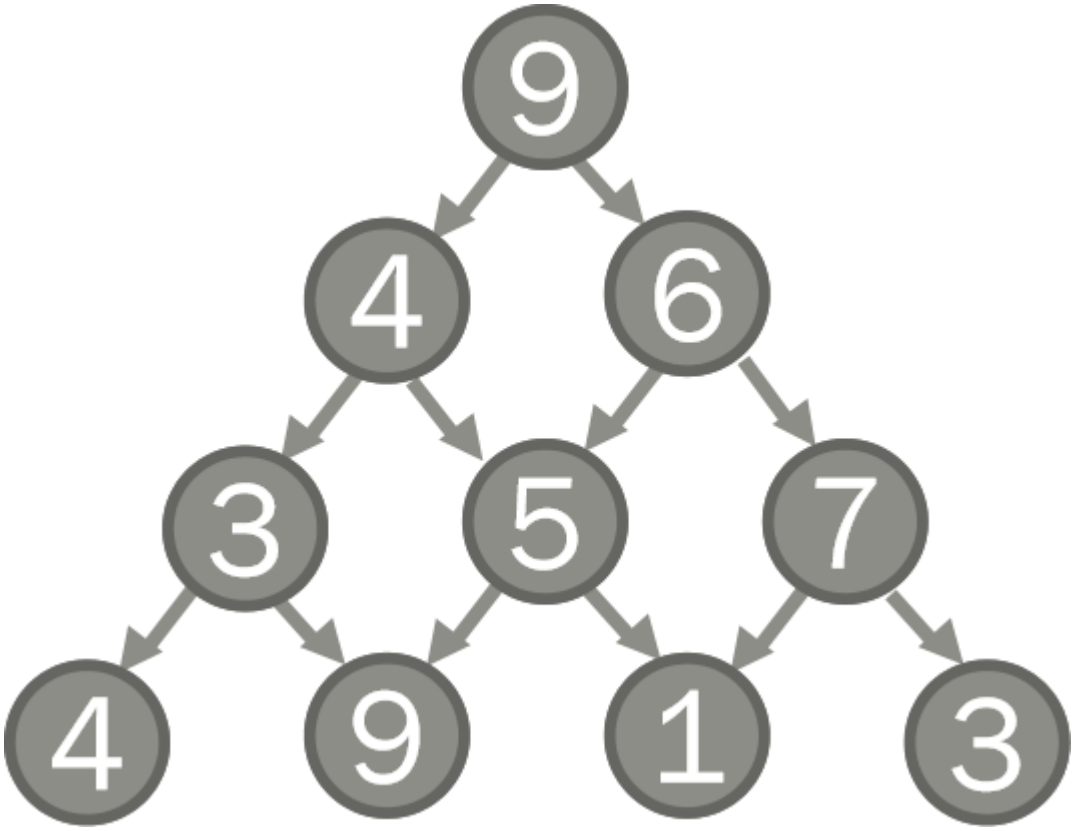
【核心代码】

```
int n=1;
while (cin>>a[n])n++;
n--;
int len=0,L,R,mid;
for(int i=1;i<=n;i++){
    if (len==0||a[i]<=b[len])b[++len]=a[i];
    else{
        L=1;R=len;
        while(L<R){
            mid=(L+R)/2;
            if (a[i]>b[mid])R=mid;else L=mid+1;
        }
        b[L]=a[i];
    }
}
cout<<len<<endl;
```

例题18-2：数字三角形（zhima 782）

【题目描述】

在如下图数字三角形中寻找一条从上往下经过数字和最大的路径，路径的每一步只能向左下或向下走。



【输入】

一个整数 n $(1 \le n \le 1000)$ 表示数字三角形的规模。

对应的数字三角形，参见样例。

【输出】

一个整数，表示最大路径和。

【样例输入】

```
4
9
4 6
3 5 7
4 9 1 3
```

【样例输出】

```
29
```

【题目分析】

和上一例题相比，元素不再是一条线上，最优值的传递发生在二维平面里的相邻元素间，那么我们只要改变一下状态数组。

状态：$dp[i][j]$ 表示从第 i 行第 j 列开始到达底端路径能取得的最大和。

状态转移：$dp[i][j] = max(dp[i+1][j], dp[i+1][j+1]) + a[i][j]$，求解的顺序可以参见下方的核心代码中循环的遍历，这也是DP区别于搜索的又一项要素，自

底向上求，哪个小问题最不需要求解就先求，最后一层的状态值就是本身，倒数第二层是本身加上底下一层，以此类推，所以求解顺序按照行降序遍历，至于列无

论是从左往右还是从右往左都是可行的。

时间复杂度 $O(n^2)$，按照上一题提到的方法，状态数量为 n^2 ，状态转移操作数为 2 ，计算时间复杂度时遇到常数乘积可以忽略，因此最终为 n^2。

【核心代码】

```
for (int i = n-1;i >= 1;--i)
        for (int j = 1;j <= i;++j)
            dp[i][j] = max(dp[i+1][j],dp[i+1][j+1])+a[i][j];
```

【思考】

如何状态描述 $dp[i][j]$ 表示从第一行元素开始到第 i 行第 j 列元素的路径最大和可行吗?核心代码将如何来写？

例题18-3： 合唱队形 (NOIP 2004 提高组)

【题目描述】

给你 n $(2 \le n \le 100)$ 个同学的身高，不能改变相对顺序，最少需要几位同学出列，使剩下的同学构成中间高两边矮的合唱队形 $(T1 < \ldots < Ti > Ti+1 > \ldots > TK(1 \le i \le K))$

【样例输入】

```
8
186 186 150 200 160 130 197 220
```

【样例输出】

```
4
```

【题目分析】

本质题意就是在一行整数序列中，找一个先升后降的最长子序列，看似一道全新的题目，其实和导弹拦截很相似，你看出来了吗？求一次以每个元素作为结尾的最

长上升子序列，再求一次以每个元素作为起点的最长下降子序列，然后相加找寻一个最大值即可。

状态：$dp1[i]$ 表示到第 i 个数为止的最长上升子序列个数，$dp2[i]$ 表示从 i 开始的最长下降子序列个数。

状态转移：

$dp1[i] = dp1[j] + 1(1 <= j <= i-1)$

$dp2[i] = dp2[j] + 1(i+1 <= j <= n)$

初值：$dp1[i] = dp2[i] = 1$

终值：$maxdp1[i] + dp2[i] - 1(1 <= i <= n)$

例题18-4：挖地雷（zhima 783)

【题目描述】

在一个地图上有 n 个地窖 $(n \leq 20)$ ，每个地窖中埋有一定数量的地雷。同时，给出地窖之间的连接路径。当地窖及其连接的数据给出之后，某人可以从任一处开

始挖地雷，然后可以沿着指出的连接往下挖（仅能选择一条路径），当无连接时挖地雷工作结束。设计一个挖地雷的方案，使某人能挖到最多的地雷。

【样例输入】

```
5
10 8 4 7 6
1 1 1 0
0 0 0
1 1
1
```

【样例输出】

```
1 3 4 5
27
```

【题目分析】

和导弹拦截相比，需要输出最优值对应的方案，我们可以在状态转移时记录最优值来源，然后利用前面章节学过的递归程序进行输出。

状态：$dp[i]$ 表示挖到 i 地窖为止的最多地雷数。

状态转移：

$if(dp[i] < dp[j] + a[i])\{dp[i] = dp[j] + a[i]; c[i] = j\}$ //当两个地窖能连通时，可以在读入的时候处理到一个 $bool$ 数组中。

初值：$dp[i] = a[i](1 <= i <= n)$

时间复杂度：$O(n^2)$

输出：递归过程，参见核心代码。

【核心代码】

```
//状态转移、记录路径核心代码
for(int i=2;i<=n;i++)
        for(int j=1;j<=i-1;j++)

            if(b[j][i]){
```

```
                if(dp[i]<dp[j]+a[i]){
                    dp[i]=dp[j]+a[i];
                    c[i]=j;
                }
            }
    int max_ans=-1,k;
    for(int i=1;i<=n;i++)
        if (max_ans<dp[i]){
            max_ans=dp[i];
            k=i;
        }
//输出过程
void print_ans(int k){
    if (c[k]==0){
        printf("%d",k);return;
    }
    print_ans(c[k]);
    printf(" %d",k);
}
```

例题18-5：最长下降子序列方案数 (POJ1952)

【题目描述】

给你 N 个整数，求最大下降序列长度，并求该长度不相同的方案个数，所谓相同，是指值相同，不是编号，例如 $3\ 3\ 2$ ，最长子序列只有$3\ 2$。

【样例输入1】

```
69 68 71 70 74 73 72 67
```

【样例输出1】

```
4 1
```

【样例输入2】

```
69 68 69 68
```

【样例输出2】

```
2 1
```

【题目分析】

又是一道不仅要求最优值，并且还要记录最优方案的题目，但和挖地雷不同的是，此题还要实现方案之间的比较去重，这又如何来实现呢？

状态：$dp[i]$ 表示 i 为终点的最大长度。

状态转移：$if(a[j] > a[i]\ dp[i] = max(dp[i], dp[j] + 1)$

方案数的统计可以另加一个 c 数组 ，转移的时候赋值或累加过来，$if(dp[i] == dp[j] + 1)\ c[i] += c[j];$

相同序列如何去重呢，操作方法不唯一，这里提供一种：$if(a[i] == a[j]\ \&\&\ dp[i] == 1)\ c[i] = 0;$

【核心代码】

```
for(i=1;i<=n;i++){
     for(j=i-1;j>0;j--){
         if(a[j]>a[i]){
             if(dp[i]<dp[j]+1){
                      dp[i]=dp[j]+1;
                      c[i]=c[j];
              }
              else if(dp[i]==dp[j]+1)
                        c[i]+=c[j];
         }
         else if(a[i]==a[j])  {
            if(dp[i]==1)
               c[i]=0;
            break;
         }
     }
}
```

例题18-6：回文 (POJ1159)

【题目描述】

给你一个字符串，最少添加几个字符，使其构成回文字符串。

【样例输入】

```
5
Ab3bd
```

【样例输出】

```
2
```

【题目分析】

这道题目要能想到一个规律：原序列长度-（原序列和逆序列的最长公共子序列（简称 LCS ）长度）。

剩下就是如何利用线性DP求 LCS。

状态：$dp[i][j]$ 表示两序列分别到 i 和 j 时的最大公共长度。

状态转移：

$if(s1[i] == s2[j])\ dp[i][j] = dp[i-1][j-1] + 1;$

$else\ dp[i][j] = max(dp[i-1][j], dp[i][j-1]);$

初值：顶端左端都是0

终值：$dp[n1][n2]$

【完整代码】

```
#include<iostream>
#include<cstring>
using namespace std;
#define Max 5005
char str1[Max],str2[Max];
int maxlen[2][Max];
int main()
{
    int n,i,j;
   while(cin>>n)
   {
       for(i=1;i<=n;i++)
           cin>>str1[i];
       str1[n+1]='\0';
       for(i=1;i<=n;i++)//字符串str1的逆串
           str2[i]=str1[n-i+1];
       memset(maxlen,0,sizeof(maxlen));//初始化
       int e=0;
       for(i=1;i<=n;i++)//用动态规划求解
       {
           e=1-e;//用滚动数组，保证内存不会超出
           for(j=0;j<=n;j++)
           {
               if(str1[i]==str2[j])
                   maxlen[e][j]=maxlen[1-e][j-1]+1;
               else {
                   int len1=maxlen[e][j-1];
                   int len2=maxlen[1-e][j];
                   if(len1>len2) maxlen[e][j]=len1;
                   else maxlen[e][j]=len2;
               }
           }
       }
       cout<<n-maxlen[e][n]<<endl;
   }
}
```

例题18-7： 奶牛 (POJ2137)

【题目描述】

二维坐标系上有N $(3 \le N \le 100)$ 头牛，每头牛最多有 40 个位置可选 $(-100 \ldots 100)$，把牛按照 $1 \ldots n \ldots 1$ 的顺序用绳子串起来，求最短长度（ 结果乘以 100

取整）。

【样例输入】

```
4
1 0 0
2 1 0 2 0
3 -1 -1 1 1 2 2
2 0 1 0 2
```

【样例输出】

```
400
```

【样例解释】

1 号牛在坐标 $(0,0)$ ，2 号牛在坐标 $(1,0)$，3 号牛在坐标 $(1,1)$ ，4 号牛在坐标 $(0,1)$ 。

【题目分析】

此题把导弹拦截中的每个点化成了多个点，同时最终连线还形成一个圈，为了避免产生最后一头牛连不回它最优值来源的第一头牛位置，我们可以枚举起点第一头

牛的位置。

状态：$dp[i][j]$ 表示 第i号牛在第 j 号位为止的最短距离。

转移：$dp[i][k] = min(dp[i][k], dp[i-1][j] + dis)$ 。

初值：$dp[1][1..a[i]] = 0$ ($a[i]$ 表示第 1 头牛能在的位置数）。

终值：$max\{dp[n][..] + dis(n,1)\}$ 。

【完整代码】

```
#include <iostream>
#include <cstring>
#include <algorithm>
#include <cstdio>
#include <cmath>
#define maxn 111
#define inf 0x3f3f3f3f
using namespace std;
int n;
double dp[maxn][41];
struct P{
    int x,y;
};
double dis(P a,P b){
    return sqrt((a.x-b.x)*(a.x-b.x)+(a.y-b.y)*(a.y-b.y));
}
```

```
struct DATE{
    P p[41];
    int num;
}my[maxn];
int main(){
    while(scanf("%d",&n)!=EOF){
        for(int i=1;i<=n;++i){
            scanf("%d",&my[i].num);
            for(int j=1;j<=my[i].num;++j)
                scanf("%d%d",&my[i].p[j].x,&my[i].p[j].y);
        }
        double ans=inf;
        for(int l=1;l<=my[1].num;++l){
            memset(dp,0x7f,sizeof(dp));
            dp[1][l]=0;
            for(int i=1;i<n;++i)
                for(int j=1;j<=my[i].num;++j)
                    for(int k=1;k<=my[i+1].num;++k)
                        dp[i+1][k]=min(dp[i+1][k],dp[i]
[j]+dis(my[i].p[j],my[i+1].p[k]));
            for(int i=1;i<=my[n].num;++i)
                ans=min(ans,dp[n][i]+dis(my[1].p[l],my[n].p[i]));
        }
        printf("%d\n",int(ans*100));
    }
}
```

例题18-8：拉链 (POJ2192)

【题目描述】

两个字符串镶嵌能否形成第三个字符串，不能打乱原串顺序。

【输入】

一个整数 T ，表示数据组数。

接下来 T 行，每行是用空格隔开的三个字符串。

【输出】

对于每组数据，输出是否符合题目要求，输出格式详见样例。

【样例输入】

```
3
cat tree tcraete
cat tree catrtee
cat tree cttaree
```

【样例输出】

```
Data set 1: yes
Data set 2: yes
Data set 3: no
```

【题目分析】

状态：$dp[i][j]$ 表示 A 串前 i 个 B 串前 j 个能否形成 C 串前缀。

状态转移：

$if(can[i][j-1] == 1 \,\&\&\, a[j-1] == c[i+j-1])\ can[i][j] = 1;$

$if(can[i-1][j] == 1 \,\&\&\, b[i-1] == c[i+j-1])can[i][j] = 1;$

初值：

$can[0][i] = 1$ 当a串和c串第一个字符相同。

$can[i][0] = 1$ 当b串和c串第一个字符相同。

终值：$can[n][m]$

【完整代码】

```
#include <iostream>
#include <cstdio>
#include <cstring>
using namespace std;
bool cheat(string a,string b,string c){
    bool bo = false;
    int i,j;
    int la = a.length();
    int lb = b.length();
    int can[1024][1024] = {0};
    can[0][0] = 1;
    for(i = 1;i <= la;i++){//a串第一个和c串相同，设置初值
        if(a[i - 1] == c[i - 1]){
            can[0][i] = 1;
        }else{
            break;
        }
    }
    for(i = 1;i <= lb;i++){//b串第一个和c串相同，设置初值
        if(b[i - 1] == c[i - 1]){
            can[i][0] = 1;
        }else{
            break;
        }
    }
    for(i = 0;i <= lb;i++){
        for(j = 0; j <= la ; j++){
            if(can[i][j - 1] == 1 && a[j - 1] == c[i + j - 1]){
                can[i][j] = 1;

            }
```

```
            if(can[i - 1][j] == 1 && b[i - 1] == c[i + j - 1]){
                can[i][j] = 1;
            }
        }
    }
    if(can[lb][la] == 1){
        cout<<"yes\n";
    }else{
        cout<<"no\n";
    }
    return bo;
}
int main(){
    int n;
    int id = 1;
    string a,b,c;
    cin>>n;
    for(int i = 0;i < n;i++){
        cin>>a>>b>>c;
        bool cheat(string a,string b,string c);
        cout<<"Data set "<<(id++)<<": ";
        cheat(a,b,c);
    }
}
```

例题18-9：路径 (POJ2353)

【题目描述】

给定 $M \times N$ 的矩阵,求从第一行到最后一行经过的最小和，只能从左右下三个方向走，如果有多种方案，输出任意路径即可。

【输入】

两个空格隔开的整数 M 和 N 。

接下来 M 行描述矩阵。

【输出】

输出符合题目要求的最佳路径，只输出路径中点坐标的列号。

【样例输入】

```
3 4
10 10 1 10
2 2 2 10
1 10 10 10
```

【样例输出】

```
3
3
2
1
1
```

【题目分析】

状态好设计，关键是转移的顺序要考虑清楚。

状态：$dp[i][j]$ 表示到达第 i 行 j 列的最小和。

状态转移：上述状态根据题意可以有三种来源，$dp[i-1][j]$、$dp[i][j+1]+a[i][j]$、$dp[i][j-1]+a[i][j]$，为了确保本行都已经是最优值，上一行的要先转移过

来，又由于求得是最小值，无论接下来先从左往右传递最优值还是从右往左都是可以的，从左往右就是把 $f[i][j-1]$ 转移过来，从右往左就是把 $f[i][j+1]$ 转移

过来，转移的时候可以使用一个数组记录路径，上左右来到分别对应 $r[i][j]=j$、$r[i][j]=j-1$、$r[i][j]=j+1$。

初值：$dp[1][j]$ = $a[1][j]$

终值：$min\{dp[n][j]\}$，最优方案利用r数组记录的值进行递归输出，递归 $print(n,j)$ ，遇到 $r[n][j]==0$ 终止;遇到 $==j$，$print(n-1,j)$;否则

$print(n,r[n][j])$，详见完整代码。

【完整代码】

```
#include<iostream>
#include<fstream>
using namespace std;
int dp[101][501];
int a[101][501];
int road[101][501];
int n,m;
void print(int s,int t)
    if(road[s][t]!=0){
        if(road[s][t]==t)
            print(s-1,road[s][t]);
        else
            print(s,road[s][t]);
        cout<<road[s][t]<<endl;
    }
int main(){
    int i,j,k;
    scanf("%d%d",&n,&m);
    for(i=1;i<=n;i++)
        for(j=1;j<=m;j++)
            scanf("%d",&a[i][j]);
    for(i=1;i<=m;i++)
        dp[1][i]=a[1][i];
    for(i=2;i<=n;i++){

        for(j=1;j<=m;j++){
```

```
            dp[i][j]=dp[i-1][j]+a[i][j];
            road[i][j]=j;
        }
        for(j=2;j<=m;j++){
            if(dp[i][j]>dp[i][j-1]+a[i][j]){
                dp[i][j]=dp[i][j-1]+a[i][j];
                road[i][j]=j-1;
            }
        }
        for(j=m-1;j>=1;j--){
            if(dp[i][j]>dp[i][j+1]+a[i][j]){
                dp[i][j]=dp[i][j+1]+a[i][j];
                road[i][j]=j+1;
            }
        }
    }
    int res=dp[n][1];
    j=1;
    for(i=1;i<=m;i++)
        if(res>dp[n][i]){
            res=dp[n][i];
            j=i;
        }
    print(n,j);
    cout<<j<<endl;
}
```

例题18-10：最大总和 (POJ2479)

$1S\ 64M$

【题目描述】

给定 N 个整数序列，求不相交的两个子串最大和。

【输入】

第一行 $T\ (T \leq 30)$ ，表示数据组数。

每组数据的由两行组成，第一行是一个整数 $n(2 \leq n \leq 50000)$ 。第二行包含 n 个整数：$a_1, a_2, \ldots, a_n$ 。($|ai| \leq 10000$). 每组数据之间有一个空行。

【样例输入】

```
1

10
1 -1 2 2 3 -3 4 -4 5 -5
```

【样例输出】

```
13
```

【样例解释及说明】

{2,2,3,-3,4} and {5}，由于输入量比较大，建议使用scanf。

【题目分析】

和合唱队形处理方法类似，每个整数左边最大和 + 右边最大和，然后求最大值。

状态：

$dpL[i]$ 表示以 i 结尾的连续最大和。

$dpR[i]$ 表示以 i 开始的连续最大和。

$dpR2[i]$ 表示 i 右边的连续最大和，由 $dpR[i]$ 得到。

初值：$dpL[1] = a[1]; dpR[n] = a[n]$

终值：$max\{dpL[i-1] + dpR2[i](2 <= i <= n)\}$

【完整代码】

```
#include <iostream>
#include <cstdio>
using namespace std;
#define max(a,b) a>b?a:b
#define MAXV 50010
#define inf -10010
int lt[MAXV],rt[MAXV],a[MAXV],rtm[MAXV];
int main(){
    int t,n,i,temp;
    scanf("%d",&t);
    while(t--){
        scanf("%d",&n);
        for(i=1;i<=n;i++)
            scanf("%d",&a[i]);

        temp=inf;lt[1]=a[1];rt[n]=a[n];
        for(i=2;i<=n;i++){
            lt[i]=max(a[i],lt[i-1]+a[i]);
        }
        for(i=n-1;i>=1;i--){
            rt[i]=max(a[i],rt[i+1]+a[i]);
        }

        rtm[n]=rt[n];
        for(i=n-1;i>=1;i--)
            rtm[i]=max(rtm[i+1],rt[i]);

        int ma=inf;
        for(i=2;i<=n;i++){
            ma=max(ma,lt[i-1]+rtm[i]);
        }
```

```
        printf("%d\n",ma);
    }
}
```

例题18-11：蜂巢漫步 (POJ3036)

【题目描述】

在蜂巢的原点出发，求经过 n $(1 \le n \le 14)$ 步在回到原点的走法数，答案保证小于 2^{31} 。

【输入】

第一行一个整数 T ，表示数据组数。

接下来 T 行，每行一个整数，表示询问的步数。

【输出】

每行一个整数，表示每组输入数据对应的走法数。

【样例输入】 2 2 4 **【样例输出】** 6 90

【题目分析】

状态：$dp[k][i][j]$ 表示第 k 步，到达坐标坐标(i,j) 的方案数。

状态转移：

$x = i + dirX[d]; y = j + dirY[d]; dp[i][j][k] += dp[x][y][k-1]$

初值：$dp[0][7][7] = 1$

终值：$dp[n][7][7]$

代码实现的一些细节：首先设置坐标变化六相邻int dirX[6] = { 1, 0,-1,-1, 0, 1};int dirY[6] = { 0, 1, 1, 0,-1,-1};原点可以设置到（7,7）的位置，对于最大的n=15，刚

好就够来回。

【完整代码】

```
#include<iostream>
#include<cstdio>
#define N 15
int dp[N][N][N];
int dirX[6] = { 1, 0,-1,-1, 0, 1};
int dirY[6] = { 0, 1, 1, 0,-1,-1};
int main(){
    int i,j,k,d,x,y;
    dp[7][7][0] = 1;
    for(k=1;k<N;k++){
        for(i=0;i<N;i++){
            for(j=0;j<N;j++){
                for(d=0;d<6;d++){

                    x = i+dirX[d];
```

```
                    y = j+dirY[d];
                    if(x>=0 && x<N && y>=0 && y<N){
                        dp[i][j][k] += dp[x][y][k-1];
                    }
                }
            }
        }
    }
    int t,n;
    scanf("%d",&t);
    while(t--){
        scanf("%d",&n);
        printf("%d\n",dp[7][7][n]);
    }
}
```

例题18-12：旅行 (POJ3230)

【题目描述】

一位旅行者在城市之间旅行。既有花费也有收入。

现有 n 个城市，旅行者有 m 天的旅行时间。每天他可能会去另一个城市或留在那里并支付一些钱。当他来到一个城市时，他可以得到一些钱。即使他留在城里，

也能拿到第二天的收入。所有的收入可能每天都在变化。

旅行者总是从 1 号城市开始，求收入最多的最佳旅行方式。

【输入】

有多组数据，以 0 0 结束。

每组数据的第一行两个整数n 和 m $(n, m < 100)$ ，分别表示旅游城市数和天数。

接着是一个 $n \times n$ 的矩阵 $a[i][j]$ 表示从第 i 个城市到第 j 个城市的花费。

然后是一个 $m \times n$ 的矩阵 $b[i][j]$ 代表第 i 天在第 j 个城市 的收入。

每组输入数据后有个空行。

【输出】

一个整数，表示 m 天后的最大收益。

【样例输入】

3 3 3 1 2 2 3 1 1 3 2

2 4 3 4 3 2 3 4 2

0 0

【样例输出】

8

【题目分析】

状态：$dp[i][j]$ 表示第 i 天在第 j 个城市已获最大收益（可能为负）。

状态转移： $dp[i][j] = max(dp[i][j], dp[i-1][k] - a[k][i] + b[i][j])$

初值：$dp[1][j]$

终值：$max\{dp[m][1..n]\}$

【完整代码】

```
#include<stdio.h>
#include<string.h>
#include<algorithm>
#include<iostream>
using namespace std;
#define N 105
int dp[N][N], cost[N][N], c[N][N];
int main(){
    int n, m;
    while(~scanf("%d%d", &n, &m) && (n || m)){
        for( int i = 1; i <= n; i++ ){
            for(int j = 1; j <= n; j++ )
                scanf("%d", &cost[i][j]);
        }
        for( int i = 1; i <= m; i++ ){
            for( int j = 1; j <= n; j++ )
                scanf("%d", &c[i][j]);
        }
        memset(dp, 0, sizeof(dp));
        for (int i = 1; i <= n; i++)
            dp[1][i] = c[1][i] - cost[1][i];
        for ( int i = 2; i <= m; i++ ){
            for( int j = 1; j <= n; j++ ){
                for (int k = 1; k <= n; k++ )
                    dp[i][j] = max(dp[i][j], dp[i-1][k] - cost[k][j] + c[i][j]);
            }
        }
        int ans = 0;
        for ( int i = 1; i <= n; i++ )
            ans = ans > dp[m][i] ? ans : dp[m][i];
        printf("%d\n", ans);
    }
}
```

例题18-13：能量 (POJ2948)

【题目描述】

二维矩阵中，每个单元格可以选定一个传送方向，向左或者向上，分别有个能量值可以传送，求n*m个单元格都选定方向后能传送的最大能量值（输入量比较

大），先输入右向左的能量值，再输入下往上的能量值。

【输入】

多组数据，以 0 0 结束。

每组数据的第一行是两个整数 n 和 m $(1 \le n, m \le 500)$，分别表示行数和列数。接下来两个 $n \times m$ 两个矩阵，第一个矩阵描述从右往左传输的能量值，第二个

矩阵描述从下往上传输的能量值，$(0 \le \text{能量值} \le 1000)$。

【输出】

一个整数，表示符合题意的最大值。

【样例输入】

```
4 4
0 0 10 9
1 3 10 0
4 2 1 3
1 1 20 0
10 0 0 0
1 1 1 30
0 0 5 5
5 10 10 10
0 0
```

【样例输出】

```
98
```

【题目分析】

状态：$dp[i][j]$ 表示 (i, j) 为右下角的矩阵块最大能量传送值。

状态转移：

$dp[i][j] = max(dp[i-1][j] + suma[i][j], dp[i][j-1] + sumb[i][j]);$

其中 $suma[i][j]$ 表示 (i, j) 为右端点横线和，$sumb[i][j]$ 表示 (i, j) 为下端点竖线和。

初值默认即可。

终值：$dp[n][m]$

【完整代码】

```
#include<iostream>
#include<algorithm>
#include<stdio.h>
#include<string.h>
using namespace std;
int n,m,a[505][505],b[505][505],suma[505][505],sumb[505][505],dp[505][505];
int main()
{
```

```
    while(scanf("%d%d",&n,&m)!=EOF)
    {
        if (n==0&&m==0) return 0;
        for (int i=1;i<=n;i++)
           for (int j=1;j<=m;j++)
               scanf("%d",&a[i][j]);
        for (int i=1;i<=n;i++)
           for (int j=1;j<=m;j++)
               scanf("%d",&b[i][j]);
        for (int i=1;i<=n;i++)
           for (int j=1;j<=m;j++)
               {
               suma[i][j]=suma[i][j-1]+a[i][j];
               sumb[i][j]=sumb[i-1][j]+b[i][j];
               }
        memset(dp,0,sizeof(dp));
        for (int i=1;i<=n;i++){
           for (int j=1;j<=m;j++){
               dp[i][j]=max(dp[i-1][j]+suma[i][j],dp[i][j-1]+sumb[i][j]);
               cout<<dp[i][j]<<" ";
               }
           cout<<endl;
        }
        printf("%d %d %d %d\n",dp[0][3],dp[1][2],dp[1][3],sumb[1][3]);
        printf("%d\n",dp[n][m]);
    }
}
```

18.1 小结

|线性DP的定义：线性DP是动态规划问题中的一类问题，指状态之间有线性关系的动态规划问题

|时间复杂度：一般情况下，可以通过状态数×状态转移操作数来求取。

|线性DP特点：转移在线性的范围进行，每个状态依赖的状态比较少。

线性DP是比较基础的动态规划算法，通过本章节的学习，读者需要体会动态规划相比搜索的两大重要区别点：状态、自底向上求解，学好线性DP也有利于接下来

区间DP的理解。

练习题：

洛谷1095

洛谷1541

洛谷1868

洛谷2679

洛谷2501

洛谷3336

洛谷3558

洛谷4158

洛谷5301